Names: GIKUNDIRO Denyse 26RP00964

Nzeyimana Nelly Stella 26RP00818

Mukeshimana Tumaine 25RP21087

Niganze Muvunyi Arthur 25RP02794

Hakizimana Jean De Dieu 25RP02291

Program:
B-Tech in Electronics and Telecommunication Technology

RQF level: 8

Department:
Electrical and Electronics Engineering

Option/Trade:
Electronics and Telecommunication Technology

Module name:
Machine Learning

Academic year: 2026/2027

Semester: I

Kigali, October 20

Fruit Detection Using YOLOv8
Detecting Apples, Bananas and Oranges with Deep Learning

1. Abstract

This project develops an object detection model that locates and identifies three types of fruit in images: apple, banana, and orange. The model is based on YOLOv8n, the smallest member of the YOLOv8 family from Ultralytics, and was fine-tuned from COCO-pretrained weights on a custom fruit dataset using Google Colab with a GPU. After 50 epochs of training, the model reached a mean average precision at IoU 0.5 (mAP@0.5) of about 0.80 and a precision of about 0.76 on the validation set. The trained weights are small enough to run on ordinary hardware. The model is also delivered as an interactive Streamlit web application, and the full project is published on GitHub.

2. Introduction

Automatic fruit recognition is useful in areas such as sorting and grading in agriculture, automated checkout, inventory counting and robotic harvesting. Object detection goes beyond image classification: it finds where each object is in an image and assigns it a class label, so several fruits in one picture can be handled at once.

YOLO (You Only Look Once) models perform detection in a single pass through the network, which makes them fast enough for real-time use. This project uses YOLOv8n because it offers a good balance between accuracy and speed with a very small model size.

2.1 Objectives

- Train an object detection model that recognizes apple, banana and orange.
- Evaluate the model with standard detection metrics.
- Package the trained model and code so that anyone can run it on new images.
- Build a simple web application with Streamlit so that non-programmers can try the model.
- Publish the project on GitHub so that it can be reproduced and shared.

3. Methodology

3.1 Model

The model is YOLOv8n (Ultralytics version 8.4.197), an anchor-free detector with a convolutional backbone, a feature-fusion neck and a decoupled detection head. Training started from the pretrained yolov8n.pt weights (transfer learning), so the network already knew general visual features and only had to adapt to the three fruit classes.

3.2 Dataset

The dataset is in YOLO format: each image has a text file listing, for every object, the class number and a normalized bounding box. The three classes are apple (0), banana (1), and orange (2).

The images and labels were stored in a folder named fruit_dataset and described by a data.yaml file. The data was divided into a training set, used to fit the model, and a validation set, used after every

epoch to measure performance on images the model had not trained on. All results in this report are measured on the validation set.

3.3 Training Environment and Settings

Training was run on Google Colab using a GPU. The main settings are listed in Table 1.

Table 1. Training configuration

Setting	Value
Base model	YOLOv8n (pretrained)
Task	Object detection, 3 classes
Epochs	50 (early-stopping patience 15)
Batch size	16
Image size	640 × 640
Optimizer	AdamW
Initial learning rate	0.001429 (final factor 0.01)
Warm-up epochs	3
Mosaic augmentation	On, closed for the last 10 epochs
Other augmentation	HSV colour shifts, horizontal flip, translate 0.1, scale 0.5
Mixed precision	Enabled (AMP)
Random seed	0

3.4 Training Procedure

- Install Ultralytics and check that a GPU is available.
- Load the dataset and create the data.yaml file listing the paths and class names.
- Load the pretrained yolov8n.pt weights.
- Train for 50 epochs; the best weights are saved automatically as best.pt.
- Validate the best weights and run predictions on sample images.

Training the full 50 epochs took only about 200 seconds on the GPU, which shows how lightweight the model is.

4. Results

4.1 Evaluation Metrics

Four standard metrics are used. Precision is the share of predicted detections that are correct. Recall is the share of real objects that the model finds. mAP@0.5 is the mean average precision when a

detection counts as correct if its overlap (IoU) with the true box is at least 0.5. mAP@0.5:0.95 averages this over stricter thresholds from 0.5 to 0.95 and is a tougher measure of box accuracy.

Table 2. Validation results of the best model

Metric	Value
Precision	0.761
Recall	0.708
mAP@0.5	0.798
mAP@0.5:0.95	0.656
Validation box loss	0.540
Validation class loss	0.833
Validation DFL loss	1.130

4.2 Discussion

An mAP@0.5 close to 0.80 means the model locates and classifies most fruits correctly. Precision (0.76) and recall (0.71) are fairly balanced, so the model neither produces many false detections nor misses a large share of fruits. The training box loss fell steadily over the 50 epochs, showing that the model learned the task.

The gap between mAP@0.5 (0.80) and mAP@0.5:0.95 (0.66) shows that the model finds fruits reliably but its bounding boxes are not always very tight.

4.3 Training Curves

Figure 1 shows how the losses and metrics changed over the 50 training epochs. The training losses decrease steadily. The validation losses are unstable in the first few epochs, which is normal when a pretrained model is adapted to new classes, and then settle and follow the training losses closely. The small gap between them at the end suggests that the model did not overfit strongly.

Precision, recall and both mAP values rise throughout training and level off near the end, with the best result at epoch 48 (mAP@0.5 = 0.798, mAP@0.5:0.95 = 0.656). The precision and recall curves fluctuate more than the mAP curves, which is normal because these two values depend on the confidence threshold and can change from one epoch to the next.

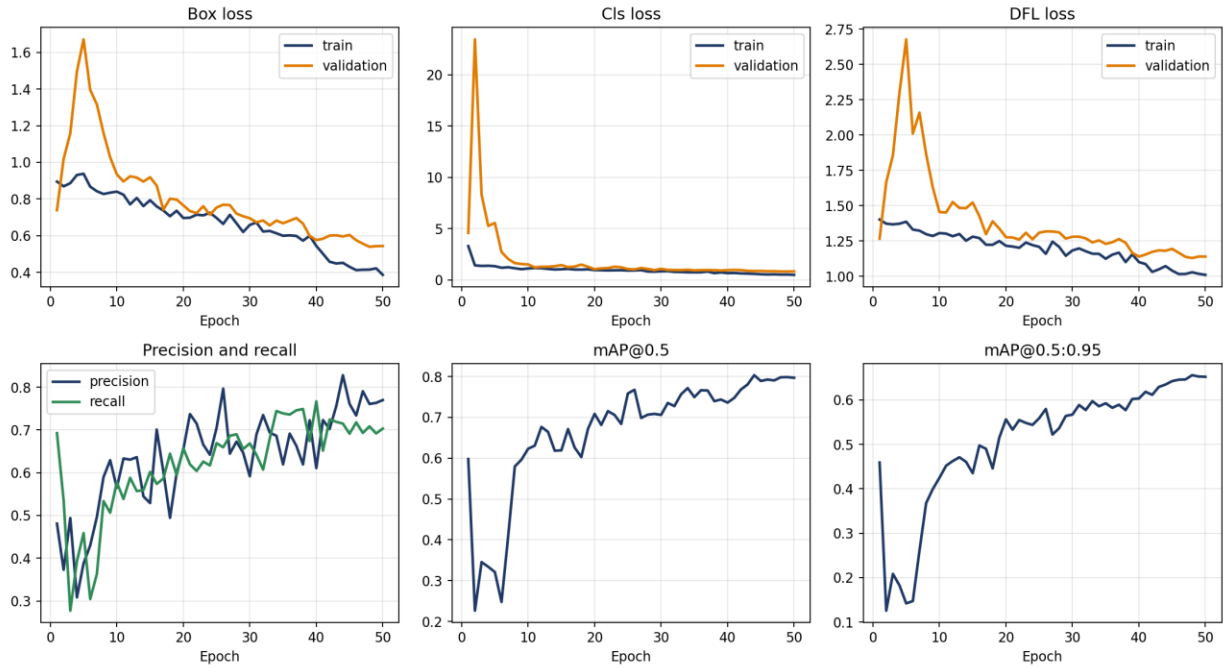


Figure 1. Training and validation curves over 50 epochs

5. Web Application (Streamlit)

To make the model easy to use without writing code, a web application was built with Streamlit, an open-source Python framework for data and machine-learning apps. The application is contained in a single file, app.py.

5.1 Features

- Upload an image (JPG, PNG or WebP) from the computer.
- Adjustable confidence threshold slider in the sidebar.
- Side-by-side display of the original image and the image with detected bounding boxes.
- Count of each fruit type found, plus a table listing every detection with its confidence score.
- A warning message when no fruit is detected, suggesting a lower threshold.

5.2 How It Works

When the app starts, it loads best.pt once and keeps it in memory (cached) so that later predictions are fast. When the user uploads an image, the app runs the YOLOv8 model, draws the boxes on the image, converts the colours for display, and shows the results with the counts.

5.3 Running the App

```
pip install -r requirements.txt
streamlit run app.py
```

The app then opens in the browser at http://localhost:8501. It can also be deployed for free on Streamlit Community Cloud by connecting the GitHub repository and selecting app.py as the main file.

Live demo: https://fruit-detection-system-awuxgmavrzowle5qipmsxd.streamlit.app/

Figure 2 shows the deployed application detecting fruit in an uploaded photo. The confidence threshold slider is visible on the left, the original image and the detection result are shown side by side, and each box is labeled with the predicted fruit and its confidence score.

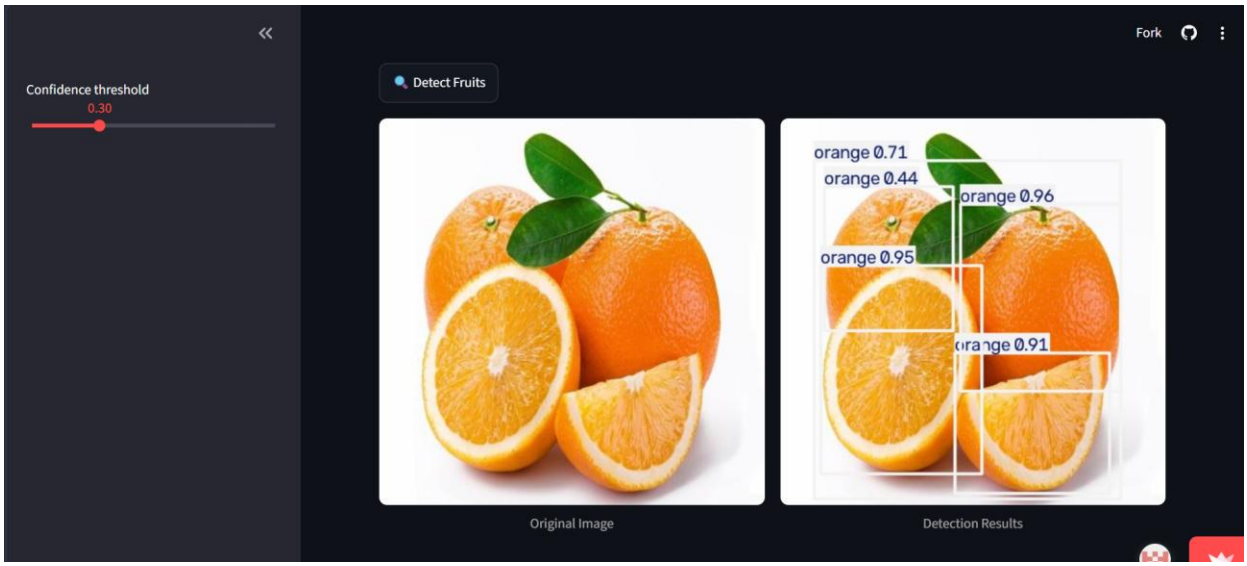


Figure 2. Streamlit web application detecting oranges (confidence threshold 0.30)

Figure 3 shows a second example. The app found five bananas and displays the total count for each fruit type, followed by the confidence of every individual detection.

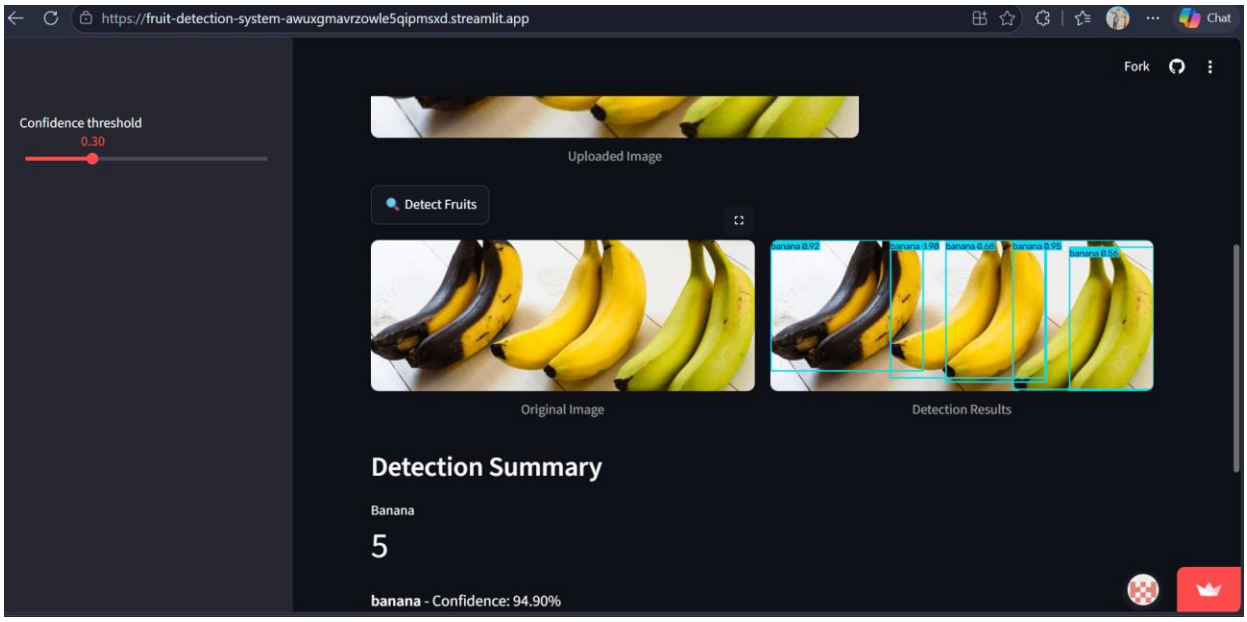


Figure 3. Banana detection with fruit count summary on the deployed app

6. Using the Model from the Command Line

The trained weights (best.pt) can also be used directly with a short script. To detect fruit in an image:

```
pip install ultralytics==8.4.197
python predict.py path/to/image.jpg
```

The script prints the class and confidence of every detection and saves an annotated copy of the image. A Google Colab demo notebook is also included.

7. Project Repository (GitHub)

The complete project is published in a public GitHub repository so that others can view, download and reproduce the work.

Repository: https://github.com/gikundirodenise14-dotcom/fruit-detection-system

7.1 Repository Contents

File	Purpose
app.py	Streamlit web application
best.pt	Trained YOLOv8n model weights
predict.py	Command-line script for detection
demo_notebook.ipynb	Google Colab demo of the trained model
training_notebook.ipynb	Training steps and settings
requirements.txt	Python packages (Ultralytics 8.4.197, Streamlit, etc.)

File	Purpose
packages.txt	System package needed for Streamlit Cloud deployment
README.md	Project description and instructions

7.2 Publishing Steps

- Create a new empty repository on GitHub.
- Initialize Git in the project folder and commit all files (git init, git add ., git commit).
- Connect the local folder to the GitHub repository (git remote add origin) and push (git push -u origin main).
- Optionally deploy the app on Streamlit Community Cloud from the repository.

The model file is about 6.5 MB, well below GitHub's 100 MB file limit, so it can be stored in the repository directly without Git LFS.

8. Limitations and Future Work

- The model knows only three fruits; other fruits will be missed or mislabeled.
- Accuracy may drop with unusual lighting, heavy occlusion or cluttered backgrounds that are not represented in the training data.
- Performance could be improved with a larger and more varied dataset, longer training, or a bigger model such as YOLOv8s or YOLOv8m.
- Adding more fruit classes, webcam or video input in the Streamlit app, and a mobile version are natural next steps.

9. LINKS

Live demo: https://fruit-detection-system-awuxgmavrzowle5qipmsxd.streamlit.app/

GitHub: https://github.com/gikundirodenise14-dotcom/fruit-detection-system

10. Conclusion

A YOLOv8n detector was successfully trained to recognize apples, bananas and oranges, reaching an mAP@0.5 of about 0.80. The small model trains in minutes and runs quickly. The Streamlit web application lets anyone test it from a browser, and the GitHub repository makes the project easy to reproduce and share. The results show that transfer learning with a compact YOLO model is an effective approach for simple fruit detection tasks.

11. References

- Jocher, G., Chaurasia, A., Qiu, J. Ultralytics YOLO. https://docs.ultralytics.com

- Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.
- Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV.
- Streamlit Inc. Streamlit documentation. https://docs.streamlit.io